Lab 5. User Management

Content

Managing user accounts

About user accounts

Before we can do something in Oracle, we need to have a user ID created to enable us to log into Oracle. As a DBA, we will begin with the SYS or SYSTEM accounts since these accounts both have the DBA role and exist in all Oracle databases. They are often used to perform database administration tasks. The SYS account is also granted the SYSDBA privilege and is the schema in which the Oracle catalogue is stored. We should only use the SYS account when we need to perform a task as SYS or need the SYSDBA privilege.

Create a user

We can create a user using the CREATE USER command or using the Oracle Enterprise Manager tool. In order to create a user, you will need to decide the following:

- The **initial password** that the user will log in with. The user must change this during the first logon if we choose to expire the password in advance of the user's first logon.
- **Privileges** to grant to the user can be direct privileges or roles.
- The **default tablespace** where segments created by this user will be placed unless a tablespace name is used in the DDL to override this.
- A **temporary tablespace** to store internal data used by Oracle while queries are running.
- Whether to **expire the password** so that the user needs to change it the first time they log into Oracle.
- Whether to employ user **quotas on tablespaces**, which put a limit on the amount of space that a user's objects can take up in that tablespace.

- The **authentication type** allows us to specify whether you want Oracle to force the user to specify a password when they log in, or we can trust the operating system to do this for us.
- A **profile** for the user, which can be employed to manage the user's session by limiting resources that sessions can use and that help implement corporate password policies.
- The **account status** of the user (lock or unlock) as it is created.

When we create a user, we are also implicitly creating a **schema** for that user. A schema is a **logical container** for the **database objects** (such as tables, views, triggers, and so on) that the user creates. The schema name is the same as the user name, and can be used to unambiguously refer to objects owned by the user. For example, HR.EMPLOYEES refers to the table named EMPLOYEES in the HR schema (the EMPLOYEES table is owned by HR user who owns the HR schema). The terms database object and schema object are used interchangeably.

When we drop (delete) a user, we must either first drop all the user's schema objects, or use the cascade feature of the drop operation, which simultaneously drops a user and all of that user's schema objects.

A simple syntax of the CREATE USER command is as follows:

```
CREATE USER username
    IDENTIFIED BY password
    [DEFAULT TABLESPACE tablespace]
    [QUOTA {size | UNLIMITED} ON tablespace]
    [PROFILE profile]
    [PASSWORD EXPIRE]
    [ACCOUNT {LOCK | UNLOCK}];
```

- **DEFAULT TABLESPACE**: This clause specifies the default tablespace for objects created by the user. Otherwise, such objects are stored in the default tablespace of the database. If no default tablespaces are specified for this particular database, the objects will get into the system tablespace.
- **QUOTA**: This clause specifies how much space this user can allocate in the tablespace. Multiple QUOTA clauses in one Oracle CREATE USER command can be present if you need to specify several tablespaces. The clause can include the UNLIMITED definition to allow this definite user to allocate the tablespace as much as needed, without bounds. The QUOTA clause does not apply to temporary tablespaces.
- **PROFILE**: This optional clause lets you limit the database resources for this specific user at once when the particular profile defines the limitations. Without this clause, a new user automatically comes under the default profile.
- **PASSWORD EXPIRE**: The user must change the password when he/she tries to log into the database for the first time.
- **ACCOUNT LOCK/ACCOUNT UNLOCK**: We may use one of these clauses. With LOCK applied, Oracle creates the user account, but that account won't have access to the database. If we

apply the UNLOCK clause or don't specify any of these two clauses, the account will be usable at once. The unlocked status is the default.

Examples of creating users:

- **Example 1**: Create a user with a username and password

```
CREATE USER usr1 IDENTIFIED BY usr1;
```

- **Example 2**: Create a user with a default tablespace

```
CREATE USER usr2 IDENTIFIED BY usr2
     DEFAULT TABLESPACE users;
```

Note: To view a list of tablespace, use one of the following statements.

```
SELECT tablespace_name FROM user_tablespaces;
```

Or,

```
SELECT tablespace_name FROM dba_tablespaces;
```

- **Example 3**: Creates a user and specifies the user *password*, *default tablespace*, *temporary tablespace* where temporary segments are created, and *tablespace quotas*.

```
CREATE USER jward
     IDENTIFIED BY password
     DEFAULT TABLESPACE data_ts
     QUOTA 100M ON test_ts
     QUOTA 500K ON data_ts
     TEMPORARY TABLESPACE temp_ts
```

To check all users inside database:

```
SELECT username, account_status, default_tablespace
     FROM dba_users;
```

To check current user:

```
SHOW user;
```

Assigning a default tablespace for the user

Each user should have a default tablespace. The default setting for the default tablespaces of all users is the SYSTEM tablespace. If a user does not create objects, and has no privileges to do so, then this default setting is fine. However, if a user is likely to create any type of object, then we should specifically assign the user a default tablespace, such as the USERS tablespace. Using a

tablespace other than SYSTEM reduces contention between data dictionary objects and user objects for the same data files. In general, do not store user data in the SYSTEM tablespace.

We can use the CREATE TABLESPACE SQL statement to create a permanent default tablespace other than SYSTEM at the time of database creation, to be used as the database default for permanent objects.

About the authentication

Authentication means verifying the identity of someone who wants to use data, resources, or applications. After authentication, **authorization** processes can allow or limit the levels of access and action permitted to that entity. We can authenticate both database and non-database users for an Oracle database.

To **create passwords for users**, we use the IDENTIFIED BY clause of the CREATE USER or ALTER USER statements. The following example shows several SQL statements that create passwords with the IDENTIFIED BY clause:

```
CREATE USER psmith IDENTIFIED BY password;
GRANT CREATE SESSION TO psmith IDENTIFIED BY password;
ALTER USER psmith IDENTIFIED BY password;
```

Besides identifying by password, we can create users who are **authenticated by the operating system**. To do it, we need to include the EXTERNALLY clause in the CREATE USER command. These users can access the database after getting authenticated by Windows without entering other passwords. Working under the external user is a standard option for **regular database users**.

The following statement create a new external user for our database. The name is *external_user1*, *no password* is needed, the default tablespace *tbs_new_10* with a quota of *10Mb* is assigned:

```
CREATE USER ops$external_user1
   IDENTIFIED EXTERNALLY
   DEFAULT TABLESPACE tbs_new_10
   QUOTA 10M ON tbs_new_10;
```

Alter a user

Altering a user means changing some of that user's attributes. We can change all user attributes except the user name, default tablespace, and temporary tablespace. If we want to change the user name, we must drop the user and re-create that user with a different name (before dropping a user, we must ensure that the user's schema objects are either no longer needed or are backed up.

Users can change their own passwords. However, to change any other option of a user security domain, we must have the ALTER USER system privilege. We can alter user security settings with the ALTER USER statement. Changing user security settings affects the future user sessions, not current sessions.

- Change password:

```
ALTER USER dolphin IDENTIFIED BY xyz123;
```

- Lock/Unlock a user:

```
ALTER USER dolphin ACCOUNT < LOCK | UNLOCK >;
```

- Set the user's password expired:

```
ALTER USER dolphin PASSWORD EXPIRE;
```

- Set the default profile for a user:

```
ALTER USER dolphin PROFILE ocean;
```

- Set default roles for a user:

```
ALTER USER dolphin DEFAULT ROLE super;
```

Drop a user

When we drop a user account, Oracle removes the user account and associated schema from the data dictionary. It also immediately drops all schema objects contained in the user schema, if any. Therefore, if a user schema and associated objects must remain but the user must be denied access to the database, then revoke the CREATE SESSION privilege from the user.

To drop a user, we use the DROP USER statement. Because the DROP USER system privilege is powerful, a security administrator is typically the only type of user that has this privilege. If the schema of the user contains any dependent schema objects, then use the CASCADE option to drop the user and all associated objects and foreign keys that depend on the tables of the user successfully. If we do not specify CASCADE and the user schema contains dependent objects, then an error message is returned and the user is not dropped. Syntax of the DROP USER statement is as follows:

```
DROP USER user_name [ CASCADE ];
```

Configuring privilege and role authorization

About privileges and roles

A **user privilege** is the *right to run* a particular type of SQL statement, or the *right to access an object* that belongs to another user, run a PL/SQL package, and so on. The types of privileges are defined by Oracle Database.

- System privileges: allow the grantee to perform standard administrator tasks in the database. Restrict them only to trusted users.
- Object privileges: each type of object has privileges associated with it.

Roles are created by users (usually administrators) to group together privileges or other roles. They are a way to facilitate the *granting of multiple privileges* or roles to users. We must enable the role for a user before the user can use it.

Managing system privileges

A **system privilege** is the right to perform a particular action or to perform an action on any schema objects of a particular type. For example, the privileges to create tablespaces and to delete the rows of any table in a database are system privileges. There are over 100 distinct system privileges. Each system privilege allows a user to perform a particular database operation or class of database operations. Because the system privileges are very powerful, only grant them when necessary to roles and trusted users of the database.

We can grant or revoke system privileges to users and roles using the GRANT or REVOKE statement.

- `GRANT privilege(s) TO user/role`
- `REVOKE privilege FROM user/role`

A complete list of system privileges and their descriptions are described in the following link: https://docs.oracle.com/cd/E11882_01/server.112/e41084/statements_9013.htm#SQLRF01603

Managing user roles

Roles help to manage and control the privileges more easily. They are named groups of related privileges that we grant as a group to users or other roles. Within a database, each role name must be unique and different from all user and other roles. Unlike schema objects, roles are not contained in any schema. Therefore, a user who creates a role can be dropped with no effect on the role.

Roles are useful for quickly and easily granting permissions to users. Oracle provides many pre-defined roles, and we can also create our own roles.

Roles have the following functionality:

- A role can be granted system or object privileges.
- Any role can be granted to any database user.
- A role can be enabled or disabled.
- A role can be granted to other roles. However, a role cannot be granted to itself and cannot be granted circularly. For example, role r1 cannot be granted to role r2 if role r2 has previously been granted to role r1.
- We enable or disable the *default role* of a directly granted role by using the `DEFAULT ROLE` clause of the `ALTER USER` statement.

In general, we create a role to serve one of two purposes:

- To manage the privileges for a database application.
- To manage the privileges for a user group.

Oracle pre-defined roles:
https://docs.oracle.com/cd/E11882_01/network.112/e36292/authorization.htm#i1007091

We can create a role using the CREATE ROLE statement.

```
CREATE ROLE role_name [ IDENTIFIED BY password ]
```

We must have the CREATE ROLE system privilege to do so. Typically, only security administrators have this system privilege. After you create a role, the role has no privileges associated with it. To use the GRANT statement, we need to grant either privileges or other roles to the new role.

We must **enable** the role for a user before the user can use it using the SET ROLE statement:

```
SET ROLE role_name IDENTIFIED BY role_password
```

To grant roles to users and roles, we use the GRANT statement. A user must be granted the role with the ADMIN option or was granted the GRANT ANY ROLE system privilege. For example, the following statement grants the role new_dba to a user usr1:

```
GRANT new_dba TO usr1
```

If you specify the WITH ADMIN OPTION clause when you grant a privilege or role to a user or role, then the privilege grant has the following expanded capabilities:

- The grantee can grant or revoke the system privilege or role to or from any other user or role in the database. Users cannot revoke a role from themselves.
- The grantee can grant the system privilege or role with the ADMIN option.
- The grantee of a role can alter or drop the role.

When a user creates a role, the role is automatically granted to the creator with the ADMIN option.

Managing object privileges

An object privilege is a right that we grant to a user on a database object. Some examples of object privileges include the right to:

- Use an edition
- Update a table
- Select rows from another user's table
- Execute a stored procedure of another user

Object privileges can be granted to and revoked from users and roles. If we grant object privileges to roles, then we can make the privileges selectively available. We can grant or revoke object privileges to or from users and roles using the GRANT and REVOKE statements.

A user automatically has all object privileges for schema objects contained in his or her schema. A user with the GRANT ANY OBJECT PRIVILEGE can grant any specified object privilege to another user with or without the WITH GRANT OPTION clause of the GRANT statement. A user with the

GRANT ANY OBJECT PRIVILEGE can also use that privilege to revoke any object privilege granted by the object owner or by some other user with the GRANT ANY OBJECT PRIVILEGE privilege. Otherwise, the grantee can use the privilege, but cannot grant it to other users. For example, the following statement grants the privilege INSERT on the table SCOTT.EMP to user USR1.

```
GRANT INSERT ON SCOTT.EMP TO USR1;
```

The following table describes the privileges corresponding to the basic DB objects.

Object	Privilege
Table	DELETE, INSERT, SELECT, UPDATE
View	SELECT, INSERT, UPDATE, DELETE
Procedure	EXECUTE

Managing resources with profiles

A **profile** is a named *set of resource limits* and password parameters that restrict database usage and instance resources for a user. You can assign a profile to each user, and a default profile to all others. Each user can have only one profile, and creating a new one supersedes an earlier version.

Profiles can be assigned only to users, not roles or other profiles. Profile assignments do not affect current sessions. Instead, they take effect only in subsequent sessions.

If a user is assigned a profile, that user cannot exceed these limits. To specify resource limits for a user, we must:

- Enable resource limits dynamically with the ALTER SYSTEM statement or with the initialization parameter RESOURCE_LIMIT. This parameter does not apply to password resources. Password resources are always enabled.
- Create a profile that defines the limits using the CREATE PROFILE statement
- Assign the profile to the user using the CREATE USER or ALTER USER statement

The following statement creates the app_usr2 profile with password limit values set:

```
CREATE PROFILE app_user2 LIMIT
   FAILED_LOGIN_ATTEMPTS 5          --max failed attempts number
   PASSWORD_LIFE_TIME 60            --pwd will expire after 60 days
                                    -- from the last change
   PASSWORD_REUSE_TIME 60           --The user cannot reuse any password that
                                    -- has been used in the past 60 days.
   PASSWORD_REUSE_MAX 5             --The user must change the password at
                                    -- least 5 times before they can reuse
                                    -- a previously used password.
   PASSWORD_VERIFY_FUNCTION
        ora12c_verify_function      -- password requirement function

   PASSWORD_LOCK_TIME 1/24          -- The acc will be locked for 1/24 of
                                    --  a day (i.e., 1 hour) after reaching
```

```
                                    --  the max no of failed login attempts
    PASSWORD_GRACE_TIME 10          -- The user will be warned 10 days before
                                    --  their password expires,
    INACTIVE_ACCOUNT_TIME 30;       -- If the user does not log in for 30
                                    --  days, the account will be locked
```

The list of Oracle password requirement functions:

https://docs.oracle.com/en/database/oracle/oracle-database/12.2/dbseg/configuring-authentication.html

To drop a profile, you must have the DROP PROFILE system privilege. You can drop a profile (other than the default profile) using the SQL statement DROP PROFILE. To drop a profile currently assigned to a user, we use the CASCADE option.

Exercises

1. Create a user named SCOTT with the password as the same username.
2. List all users in the system.
   (Hint: using dba_user view)
3. Create an external user (operating system) and connect using that user.
   (Hint: A user needs the CREATE SESSION privilege to connect to the Oracle DB)
4. Create a user SHIMON with the following options:
   - Password: SHIMON
   - Default tablespace: users
   - Temporary tablespace: temp
   - Quota on the users tablespace: 10MB
   - Quota on example tablespace: unlimited
   - Account status: lock and expired
5. Connect to the database using the SHIMON account.
   (Hint: You must unlock and grant the CREATE SESSION to the user SHIMON before connecting)
6. Create a table ACCOUNT with two fields: acc_no (primary key) and balance.
7. Create a role BASIC with the following privileges and grant this role to the user SHIMON:
   - CREATE TABLE
   - RESOURCE
   - DROP ANY TABLE
   - UPDATE on the table HR.DEPARTMENTS
8. Create a profile *dev_prof* with the following resource limits and change the profile of user SHIMON to the *dev_prof* profile:
   - A user can only open a maximum of 2 sessions at the same time.
   - If a user enters the wrong password 5 consecutive times, their account will be locked.
   - A user must change at least 4 different passwords before they can reuse a previously used password.